

std::scoped_lock: Constructed with one or more mutexes. Locks all mutexes on construction. Unlocks all mutexes on destruction. **Does not support:** Deferred locking - delaying locking until after construction. Early unlocking - unlocking before destruction. Ownership transfer - using std::move to change who owns the associated mutexes.

std::unique_lock: Constructed with one mutex. Locks mutex on construction (default), or locking can be deferred. Allows unlocking and relocking. Unlocks mutex on destruction if still held. Supports transfer of ownership to a distinct unique owner via std::move. Does not support management of multiple mutexes.

std::scoped_lock is more efficient: use it unless you require the features it does not support

Condition variables: Use std::condition_variable to allow threads to wait on one another.

```
#include <condition_variable>
```

```
// Declare a condition variable; usually a class member
```

```
std::condition_variable condition_;
```

```
// Notify one or all threads waiting on the condition variable
```

```
// notify_one for 1 wait predicate; notify_all for more predicates
```

```
condition_.notify_one(); condition_.notify_all();
```

To wait on a condition variable:

- Associate a mutex with a condition variable (Multiple conditions might share the same mutex)